L37 — Binary Search Tree (BST): Search, Insert, Delete

Unit: 3 | **Chapter:** Trees | **BT Level:** BT6 | **CO:** CO2, CO4

Learning Objectives

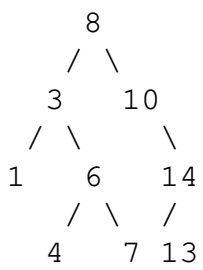
- Define BST property and verify it for a given tree
- Implement search, insert, and delete operations
- Trace deletion for all three cases
- Analyze time complexity for balanced vs degenerate BSTs

---

1. BST Property

A **Binary Search Tree (BST)** is a binary tree where for every node N:

- All values in the **left subtree** $<$ N.data
- All values in the **right subtree** $>$ N.data



BST property verified:

- Left of 8: {3, 1, 6, 4, 7} — all < 8 ✓
- Right of 8: {10, 14, 13} — all > 8 ✓

---

2. Node and BST Class

```
class BSTNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None
```

---

3. Search

```
def search(self, target):
    """Search for target. Returns node or None. O(h)"""
    node = self.root
    while node:
        if target == node.data:
            return node
        elif target < node.data:
            node = node.left
        else:
            node = node.right
    return None

# Recursive version
def search_recursive(root, target):
    if root is None or root.data == target:
        return root
    if target < root.data:
        return search_recursive(root.left, target)
    return search_recursive(root.right, target)
```

Search trace for target=7 in tree above:

```
Start at 8:  7 < 8 → go left
At 3:        7 > 3 → go right
At 6:        7 > 6 → go right
At 7:        7 == 7 → FOUND ✓
```

---

4. Insert

```
def insert(self, data):
    """Insert new value maintaining BST property. O(h)"""
    if not self.root:
        self.root = BSTNode(data)
        return

    node = self.root
    while True:
        if data < node.data:
            if node.left is None:
                node.left = BSTNode(data)
                return
            node = node.left
        elif data > node.data:
            if node.right is None:
                node.right = BSTNode(data)
                return
            node = node.right
        else:
            return   # Duplicate — no insert (or handle as per design)
```

Insert 5 into BST:

```
Start at 8:  5 < 8 → go left
At 3:        5 > 3 → go right
At 6:        5 < 6 → go left
At 4:        5 > 4 → right is None → Insert 5 as right child of 4
```

---

5. Delete (Three Cases)

Deletion is the most complex BST operation.

Case 1: Node is a Leaf (no children)

Simply remove it.

Case 2: Node has One Child

Replace node with its single child.

Case 3: Node has Two Children

Replace node's data with the **in-order successor** (smallest value in right subtree), then delete the in-order successor (which has at most one child).

```
def delete(self, data):
    """Delete node with given data. O(h)"""
    self.root = self._delete(self.root, data)

def _delete(self, node, data):
    if node is None:
        return None

    if data < node.data:
        node.left = self._delete(node.left, data)
    elif data > node.data:
        node.right = self._delete(node.right, data)
    else:
        # Node found — handle three cases
        if node.left is None:
            return node.right    # Case 1 or 2 (no left child)
        if node.right is None:
            return node.left     # Case 2 (no right child)

        # Case 3: two children
        # Find in-order successor (minimum of right subtree)
        successor = self._min_node(node.right)
        node.data = successor.data           # Replace data
        node.right = self._delete(node.right, successor.data)  # Delete

    return node
```

```
def _min_node(self, node):
    """Returns node with minimum value in subtree."""
    while node.left:
        node = node.left
    return node
```

---

6. Deletion Traces

Delete 6 (two children: 4 and 7)

```
Tree before:
       8
      / \
     3   10
    / \
   1   6
      / \
     4   7

In-order successor of 6: minimum of right subtree of 6 = 7
Replace 6's data with 7 → delete 7 from right subtree

Tree after:
       8
      / \
     3   10
    / \
   1   7
      /
     4
```

Delete 3 (two children: 1 and 6)

```
In-order successor of 3: minimum of right subtree = 4 (after above)
Replace 3's data with 4 → delete 4 from right subtree
```

---

7. In-Order Traversal = Sorted Output

```
def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.data] + inorder(root.right)

# For BST above: [1, 3, 4, 6, 7, 8, 10, 13, 14]  (sorted!)
```

This is a key BST property: **inorder traversal of a BST gives elements in sorted order**.

---

8. Minimum and Maximum

```
def find_min(root):
    """Leftmost node = minimum."""
    while root.left:
        root = root.left
    return root.data

def find_max(root):
    """Rightmost node = maximum."""
    while root.right:
        root = root.right
    return root.data
```

---

9. Complexity Analysis

Operation	Best/Average (Balanced)	Worst (Degenerate/Skewed)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Min/Max	O(log n)	O(n)
Inorder	O(n)	O(n)

Why worst case is O(n):
If elements are inserted in sorted order (1, 2, 3, 4, ...), the BST becomes a right-skewed linked list with height = n-1.

Solution: Use **self-balancing BSTs** (AVL tree, Red-Black tree) to guarantee O(log n) height.

---

10. Applications

Application	How BST is Used
Dictionary/Symbol table	Ordered key-value lookup
Database indexing	Range queries, ordered access
Auto-complete / spell check	In-order traversal gives sorted words
Expression trees	Evaluate arithmetic expressions
Event-driven simulation	Priority-ordered event queue

Summary

- BST property: left subtree < root < right subtree at every node.
- **Search/Insert:** O(h) — simple comparison-based traversal.
- **Delete:** Three cases; two-child case uses in-order successor (or predecessor).
- **Inorder traversal** of BST = sorted order of all elements.
- Performance degrades to O(n) for skewed trees → use balanced BSTs (AVL/RB-tree) for guaranteed O(log n).

References

- Cormen et al., *Introduction to Algorithms*, Ch. 12 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.4 (Springer, 2020)